

学校代码：10022

本科毕业论文（设计）

面向工具调用智能体的安全防火墙 Web 应用系统设计与实现

**Design and Implementation of a Web-based Agent Firewall for
Tool-Calling AI Systems**

霍玮放

学 院	工学院
专 业	电气工程及其自动化（辅修）
班 级	电气 24-3
学 号	230206108
指导教师	杨波 教授

2026 年 5 月

独创性声明

本人声明所呈交的论文（设计）是本人在导师指导下独立进行的设计、研究工作及取得的设计、研究成果。尽我所知，除了论文（设计）中特别加以标注和致谢的地方外，论文（设计）中不包含其他人已经发表或撰写过的研究成果，本论文（设计）中没有抄袭他人研究成果和伪造数据等行为。与我共同工作的人员对本研究所做的任何贡献均已在论文（设计）中作了明确的说明并表示了谢意。

作者签名：

日期： 年 月 日

关于毕业论文（设计）使用授权的说明

本人完全了解北京林业大学有关保留、使用毕业论文（设计）的规定，即：本科生在校期间毕业论文（设计）工作的知识产权单位属北京林业大学；学校有权保留并向国家有关部门或机构送交论文（设计）的纸质版和电子版，允许毕业论文（设计）被查阅、借阅和复印；学校可以将毕业论文（设计）的全部或部分内容公开或编入有关数据库进行检索，可以允许采用影印、缩印或其它复制手段保存、汇编毕业论文（设计）。

作者签名：

指导老师签名：

日 期： 年 月 日

AI 工具规范使用诚信承诺书

一、AI 工具使用情况说明

本人郑重声明，在毕业论文（设计）写作过程中使用 AI 工具的情况如下：

☐ 未使用任何 AI 工具（勾选此项者无需填写后续内容）

☒ 使用 AI 工具（勾选此项者需如实填写以下内容）

- 使用工具名称：ChatGPT、DeepSeek、文献数据库检索工具、LaTeX 排版与图表辅助工具。
- 具体用途及生成内容：用于文献检索关键词整理、论文结构草拟、语言润色建议、LaTeX 格式调整、图表生成脚本辅助编写。系统设计、实验方案、核心代码理解、数据复核、结论推导由作者结合本项目实际实现独立完成。
- 生成内容占全文比例：_____ %（最终提交前由作者据实填写）。

二、诚信承诺

本人承诺：毕业论文（设计）核心研究内容（包括实验设计、数据分析、结论推导等）均独立完成，未使用 AI 工具直接生成或替代。上述 AI 工具使用情况描述真实、准确、完整，无隐瞒或虚假陈述。

作者签名：

日期： 年 月 日

摘要

大语言模型驱动的智能体正在从单轮问答系统演进为能够读取外部上下文、调用工具、创建子任务并执行跨系统操作的自动化运行时。OpenClaw、Model Context Protocol (MCP) 等协议和框架降低了模型接入工具的成本，但也把传统应用中的权限边界、输入验证、审计追踪和供应链信任问题重新暴露给自然语言接口。提示词注入、间接工具结果注入、越权工具调用、混淆代理、委派链滥用等问题表明，仅依赖模型自身遵循系统提示词不足以构成安全边界。

本研究基于现有 Agent-Firewall 项目，设计并实现了一个面向工具调用智能体的双层安全防火墙 Web 应用系统。系统由代理防火墙层与智能体运行时防护层组成：代理层以 FastAPI 和 LangGraph 构建 9 节点流水线，在模型调用前后执行解析、意图分类、规则匹配、并行扫描、风险聚合、提示词改写、输出过滤和审计记录；智能体层构建 11 节点 LangGraph 运行图，在工具执行前后实施 RBAC 权限检查、参数 Schema 验证、注入检测、上下文风险识别、会话限额、敏感工具确认、工具结果 PII/密钥清洗和子智能体委派追踪。系统前端采用 Nuxt 4 与 Vuetify 实现策略管理、攻击演练、Agent 配置、调用链可视化和 Trace 分析，后端支持 Internal、MCP 和 OpenClaw 三类工具提供者。

实验采用离线可复现口径，不依赖真实模型密钥、OpenClaw 服务或 PostgreSQL。Agent 层 125 个纯内存测试全部通过，覆盖 RBAC、前置工具门控、后置输出清洗和 Agent 图执行。红队基准包含 358 个场景，其中 338 个为攻击样本、20 个为安全样本；在当前轻量离线环境下，balanced 策略检出 249 个攻击样本，检出率为 73.7%，误报率为 0.0%。仓库历史完整扫描器基准显示，在启用 LLM Guard、NeMo Guardrails 等完整本地扫描器后，攻击检出率可达 97.9%，同样保持 0.0% 误报。本文进一步构造了 10 类 Agent/subagent 调用案例，验证系统可对正常订单查询、知识库检索、越权密钥访问、参数注入、外泄上下文、重复失败升级、敏感工具确认、子智能体创建和委派链进行可解释拦截与审计。

研究表明，将安全决策从 LLM 推理闭环中剥离，并在文本入口、工具调用前、工具调用后和最终模型调用前形成多层确定性控制，可以显著降低工具调用智能体的协议级与运行时风险。系统仍存在对新型语义伪装、多轮低慢攻击、完整生产认证与长期 Trace 持久化支持不足等局限，后续可在能力证明、工具描述签名、语义基线漂移检测和跨 Agent 委派策略验证方面继续完善。

关键词：智能体安全，工具调用，提示词注入，OpenClaw，MCP，RBAC，Agent 防火墙

Abstract

Large-language-model-based agents are evolving from single-turn chat systems into autonomous runtimes that can read external context, invoke tools, create sub-tasks, and operate across software systems. Frameworks and protocols such as OpenClaw and the Model Context Protocol (MCP) reduce the integration cost of external tools, but they also expose authorization boundaries, input validation, auditability, and supply-chain trust to natural-language interfaces. Prompt injection, indirect tool-result injection, unauthorized tool use, confused-deputy attacks, and delegation-chain abuse show that model obedience to system prompts alone cannot be treated as a reliable security boundary.

This thesis designs and implements a web-based two-level Agent Firewall for tool-calling AI systems based on the existing Agent-Firewall project. The first level is a proxy firewall implemented with FastAPI and a 9-node LangGraph pipeline. It performs request parsing, intent classification, rule matching, parallel scanning, risk aggregation, prompt transformation, output filtering, and audit logging before and after LLM calls. The second level is an agent runtime guardrail implemented as an 11-node LangGraph state machine. It enforces RBAC, JSON-schema argument validation, argument injection detection, contextual risk checks, session budgets, human confirmation for sensitive tools, PII and secret redaction in tool outputs, and sub-agent delegation tracing. The web console is built with Nuxt 4 and Vuetify, supporting policy management, attack playgrounds, agent configuration, trace visualization, and analytics. The runtime supports Internal, MCP, and OpenClaw tool providers.

The evaluation follows an offline and reproducible methodology without relying on production API keys, a running OpenClaw service, or PostgreSQL. At the agent layer, 125 in-memory tests passed, covering RBAC, pre-tool gating, post-tool sanitization, and graph execution. The red-team benchmark contains 358 scenarios, including 338 attack samples and 20 safe samples. Under the current lightweight offline setting, the balanced policy detected 249 attack samples, achieving a 73.7% detection rate and a 0.0% false positive rate. The historical full-scanner benchmark in the repository shows that, when local scanners such as LLM Guard and NeMo Guardrails are enabled, the attack detection rate reaches 97.9% while preserving a 0.0% false positive rate. In addition, this thesis constructs more than ten Agent/sub-agent call-chain cases, verifying explainable control over normal order lookup, knowledge-base search, unauthorized secret access, argument injection, data exfiltration context, repeated-block escalation, sensitive-tool confirmation, sub-agent creation, and delegated execution.

The results indicate that moving security decisions outside the LLM reasoning loop and enforcing deterministic controls at the text ingress, pre-tool, post-tool, and pre-LLM stages can substantially reduce protocol-level and runtime risks in tool-calling agent systems. Remaining limitations include novel semantic obfuscation, slow multi-turn attacks, production-

grade authentication, and long-term trace persistence. Future work should strengthen capability attestation, signed tool descriptors, semantic drift detection, and policy verification across multi-agent delegation chains.

Keywords: AI agent security, tool calling, prompt injection, OpenClaw, MCP, RBAC, Agent Firewall

目 录

主要符号与缩略词对照表

符号或缩略词	含义
Agent	具备自主规划、上下文记忆和工具调用能力的智能体运行单元。
LLM	Large Language Model，大语言模型。
MCP	Model Context Protocol，用于连接模型、上下文和工具的协议。
OpenClaw	具备技能、工具、文件系统和外部服务访问能力的智能体运行框架。
RBAC	Role-Based Access Control，基于角色的访问控制。
PII	Personally Identifiable Information，个人身份信息。
Pre-tool gate	工具调用前置门控，用于判定角色是否允许调用工具以及参数是否安全。
Post-tool gate	工具调用后置门控，用于清洗工具输出中的 PII、密钥和间接提示词注入内容。
Trace	一次 Agent 请求的结构化审计轨迹，包括意图、工具计划、门控决策、模型调用和耗时。

1 绪论

1.1 研究背景

大语言模型在推理、代码生成和自然语言交互方面的能力提升，使应用系统从“用户输入-模型回答”的单一形态逐步转向“模型规划-工具调用-结果整合-继续执行”的智能体架构。与传统聊天机器人相比，工具调用智能体具有三个关键变化。第一，模型不再只是生成文本，而是可以触发具备读写副作用的外部能力，如查询订单、发送邮件、修改 Issue、读取文件或调用支付系统。第二，模型推理上下文不再只包含用户输入，还包含系统提示词、历史记忆、检索文档、工具描述、工具返回结果和其他智能体的委派结果。第三，系统的安全边界从传统 HTTP 接口扩展到自然语言、JSON 工具参数、协议元数据和模型内部决策路径。

MCP 规范将 Host、Client、Server 和 Authorization Server 等组件组织成通用协议，并对 OAuth 2.1、Protected Resource Metadata 和 token audience 校验提出要求；其安全章节明确指出服务器必须拒绝并非发给自身的访问令牌，否则会出现访问令牌权限混淆和混淆代理问题 [?]。OpenClaw 类框架则把工具、技能、文件系统和即时通讯式交互组合到个人智能体运行时中。近期 OpenClaw 安全研究显示，初始化、输入、推理、决策和执行阶段均可能出现跨阶段复合威胁，单点过滤难以处理间接提示词注入、技能供应链污染、记忆投毒和意图漂移 [?]。这些研究共同说明，智能体安全问题已经从“模型能否拒答危险内容”扩展为“系统能否在协议和工具能力层面证明、限制并审计模型的行为”。

在这种背景下，Agent-Firewall 项目的研究价值在于把工具调用智能体安全问题落到可运行、可观测和可测试的 Web 系统中。系统不是简单地增加一个提示词“请不要违规”，而是在模型外部建立确定性的策略执行点：用户输入先经过代理防火墙扫描；工具调用先经过角色、参数和上下文检查；工具输出进入模型前先经过 PII、密钥和间接注入扫描；最终调用模型前再由代理层进行二次扫描。这样可以将“模型建议做什么”和“系统允许做什么”分离，避免安全判断完全落入不可验证的 LLM 推理过程。

1.2 国内外研究现状

当前关于智能体安全的研究主要沿三条路线展开。第一类是协议级安全分析。MCP 相关研究指出，协议虽然统一了工具接入方式，但也带来了工具元数据投毒、能力声明不可验证、多 Server 信任传播、双向 Sampling 注入等问题。Maloyan 等提出 MCPBench 并指出能力证明缺失、Sampling 缺乏来源认证和多 Server 隐式信任传播是协议级薄弱环节 [?]。Huang 等基于 STRIDE 和 DREAD 对 MCP Host/Client、LLM、MCP Server、外部数据存储和授权服务器建模，强调工具描述中嵌入恶意指令是最常见且影响较大的客户端侧攻击 [?]。Jamshidi 等进一步讨论 Tool Poisoning、Shadowing 和 Rug Pull 三类语义攻击，并提出工具描述签名、语义审查和运行时启发式护栏的组

合防御 [?]

第二类是 Agent 运行时安全评估。Wang 等对 OpenClaw 及其变体进行系统安全评估，构造覆盖 Agent 执行生命周期的 205 个测试用例，指出 Agent 化系统的风险不只是底座模型风险，而是模型能力、工具使用、多步规划和运行时编排耦合后的系统性风险 [?]。另有研究从 Capability、Identity 和 Knowledge 三个维度分析 OpenClaw 持久状态，发现任一维度被污染都会显著提高攻击成功率 [?]。这些工作表明，工具调用系统的攻击面往往跨越输入、记忆、工具描述、用户身份和外部状态，必须以生命周期视角设计防护。

第三类是工程防护框架。常见做法包括提示词注入检测、敏感信息识别、工具调用审批、RBAC、执行沙箱、审计日志和策略阈值管理。问题在于，如果这些控制点分散在不同业务代码中，往往缺乏统一的可观测性和复现实验口径；如果只使用 LLM-as-judge，又会引入判定不可重复、成本高和被同类提示词绕过的问题。Agent-Firewall 采用“确定性安全决策优先，必要时接入本地扫描器”的工程路线，用规则、Schema、角色权限、启发式风险信号和本地模型扫描器共同形成可解释决策。

1.3 研究内容与论文结构

本文围绕当前 Agent-Firewall 项目展开，主要研究内容包括：

- (1) 构建面向工具调用智能体的威胁模型，明确用户输入、模型上下文、工具描述、工具结果、RBAC 配置和 Trace 日志等资产的敏感性。
- (2) 设计双层防火墙架构：代理防火墙负责模型调用前后的文本风险控制，Agent 运行时负责工具能力边界和委派链控制。
- (3) 实现 9 节点 Proxy pipeline 与 11 节点 Agent pipeline，将风险评分、工具门控、参数验证、输出清洗、确认流和审计追踪集成到可运行 Web 系统中。
- (4) 基于离线可复现实验评估系统，包括红队场景检出率、误报率、延迟开销、Agent 门控测试和多类 Agent/subagent 调用链案例。
- (5) 分析系统局限，并提出生产化改进方向，包括工具描述签名、能力证明、长期 Trace 存储、认证接入和多轮语义攻击防护。

论文结构如下：第 1 章介绍研究背景、相关工作和研究内容；第 2 章进行需求分析与威胁建模；第 3 章给出系统总体架构；第 4 章分析核心模块实现；第 5 章给出实验设计、数据和案例分析；第 6 章总结全文并提出展望。

2 需求分析与威胁模型

2.1 系统目标与设计约束

Agent-Firewall 的直接目标是为工具调用智能体提供一套可插拔、可观测、可复现的安全控制层。与传统 Web 应用只需校验 HTTP 请求参数不同，本系统需要同时处理自然语言输入、OpenAI 格式消息、工具调用计划、工具参数、工具返回结果、外部工具提供者和子智能体委派关系。因此，系统设计必须满足以下约束。

- (1) **安全决策确定性。**是否拦截、清洗或要求确认，应主要由规则、阈值、Schema 和角色权限决定，不能完全交给 LLM 自由判断。
- (2) **不破坏 Agent 能力。**防护层不能简单禁止所有工具调用，而应区分读工具、写工具、敏感工具、低风险检索和高风险外部操作。
- (3) **支持多工具生态。**项目需要支持内部工具、MCP 工具和 OpenClaw 技能桥接，工具来源不同但安全控制接口应统一。
- (4) **可观测与可审计。**每次请求需要记录意图、风险信号、门控决策、工具计划、工具结果清洗结果和耗时，便于复盘攻击链。
- (5) **离线可复现。**毕业论文实验不能依赖商业 API 密钥或真实外部服务，应能在本地通过 mock LLM、静态红队场景和纯内存测试复现。

2.2 资产与信任边界

表??列出了系统中的关键资产。可以看到，Agent 安全的敏感资产不只包括 API Key 和数据库凭证，还包括系统提示词、工具描述、RBAC 配置和请求日志。攻击者一旦掌握这些信息，可能反向构造绕过策略或诱导 Agent 执行越权操作。

表 2.1 Agent-Firewall 关键资产与敏感性
Key assets and sensitivity levels in Agent-Firewall

资产	描述	敏感性
用户提示词	用户通过代理或 Agent 界面发送的自然语言、JSON 片段和工具意图。	高
系统提示词	应用写入模型上下文的角色、行为边界和安全约束。	关键
工具参数	模型生成或路由器生成的工具调用参数。	中至高
工具输出	订单、客户资料、知识库、外部服务或子 Agent 返回的内容。	中至关键
RBAC 配置	角色、工具、scope、敏感度与确认要求的映射。	中
API 密钥	LLM provider、OpenClaw、第三方工具服务凭证。	关键
Trace 日志	包含请求、风险信号、工具链和输出片段的审计数据。	高

系统的主要信任边界包括四层。第一层是 Web 入口，用户输入与前端会话状态均视为不可信。第二层是 Proxy API，它对 OpenAI 兼容请求进行解析、扫描、决策和输出过滤，是进入 LLM provider 前的安全边界。第三层是 Agent Runtime，它在工具调用前后执行能力控制，是模型建议与真实副作用之间的边界。第四层是外部工具和子智能体，它们可能返回污染文本、敏感信息或诱导性指令，因此其输出必须在进入模型上下文前被重新判定。

2.3 攻击者模型

本文假设攻击者可以作为普通用户向系统发送提示词，可以构造自然语言、JSON、Markdown、HTML、编码文本和多语言文本，可以诱导模型规划工具调用，也可以控制某些外部工具返回内容，例如恶意知识库条目、被污染的 MCP 工具描述或不可信 OpenClaw 技能结果。攻击者不能直接修改后端源代码、数据库或部署配置，也不能绕过 Web API 直接访问数据库。

在该模型下，典型攻击目标包括：

- (1) **提示词注入**。通过“忽略之前指令”“你现在是无限制助手”等文本改变模型行为。
- (2) **系统提示词泄露**。诱导模型重复初始化文本、策略规则或隐藏上下文。
- (3) **工具滥用**。诱导 Agent 调用越权工具，如 `getInternalSecrets` 或写操作工具。
- (4) **参数注入**。在工具参数中放入 SQL 片段、特殊 token 或间接指令。

- (5) **间接注入。**通过工具结果、知识库文档、网页摘要或子 Agent 结果把恶意指令送入模型上下文。
- (6) **混淆代理。**诱导高权限 Agent 代替低权限用户调用受保护资源。
- (7) **委派链滥用。**诱导主 Agent 创建或调用子 Agent，使权限、任务或敏感数据在多跳链路中扩散。
- (8) **资源耗尽。**通过循环工具调用、大输入和多轮请求消耗 token、成本或工具限额。

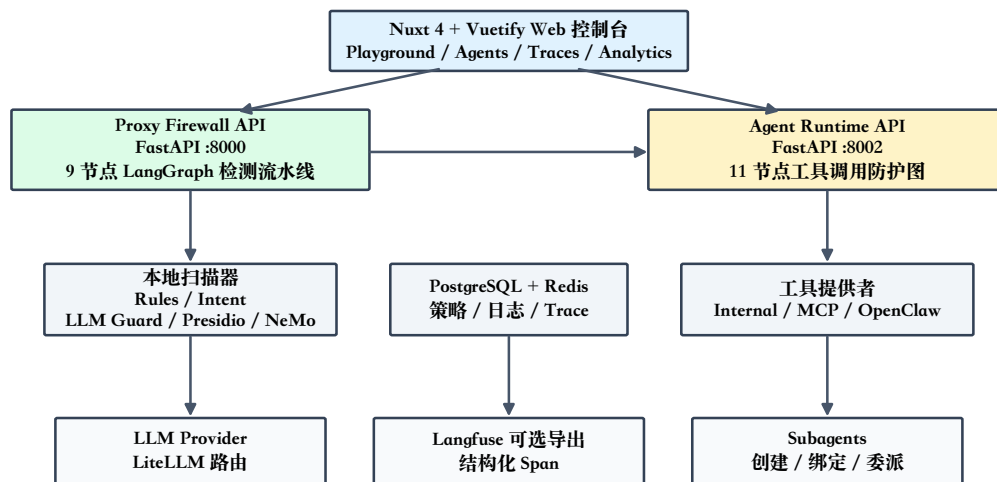
2.4 防护需求

基于上述威胁模型，系统需要在三个维度上形成防护闭环。第一是输入文本防护，即在 LLM 调用前对用户消息进行分类、规则匹配、敏感信息识别和风险评分。第二是工具能力防护，即在工具调用前确认当前角色是否拥有工具权限、参数是否符合 Schema、上下文是否具有外泄意图、会话预算是否充足以及是否需要用户确认。第三是输出与审计防护，即工具结果和模型响应中的 PII、密钥、系统提示词泄露和间接注入内容必须被清洗、阻断或记录。系统不追求“零风险”，而是追求在保留 Agent 可用性的前提下，将高风险路径转化为可解释、可复现和可审计的决策。

3 系统总体设计

3.1 总体架构

系统总体架构如图??所示。前端由 Nuxt4 和 Vuetify 构成，提供 Playground、Agent 工作区、策略管理、Trace、Analytics、Attack Scenarios 和 Settings 等页面。后端分为两个 FastAPI 服务：Proxy Service 运行在 8000 端口，负责 OpenAI 兼容模型代理、防火墙扫描、策略管理、红队运行和请求日志；Agent Service 运行在 8002 端口，负责工具调用智能体、RBAC、前后置门控、OpenClaw 桥接、子智能体委派和 Agent Trace。数据层由 PostgreSQL 和 Redis 支撑，分别用于策略、日志、Trace、配置和缓存。LLM provider 通过 LiteLLM 路由，Agent 工具提供者包括 Internal、MCP 和 OpenClaw。



设计原则：确定性安全决策与 LLM 业务推理解耦，Proxy 层管控文本风险，Agent 层管控能力边界。

图 3.1 Agent-Firewall 系统总体架构
Overall architecture of Agent-Firewall

该架构的核心思想是分层防御。Proxy 层只处理进入模型调用链路的文本和响应，不需要知道业务工具细节；Agent 层只处理工具能力边界和工具结果安全，不需要重新实现完整的文本扫描器。两层之间通过/v1/scan 和结构化风险结果衔接，使 Agent 在最终模型调用前仍能借用 Proxy 的 9 节点防火墙作为后备防线。

3.2 Proxy Firewall 9 节点流水线

Proxy Firewall 的实现位于apps/proxy-service/src/pipeline/graph.py，其 LangGraph 结构如图??所示。请求首先进入parse 节点，提取 OpenAI 格式消息、模型名、client 标识和用户消息；随后进入intent 节点进行攻击意图分类；rules 节点执行 denylist、长度、编码和特殊字符检测；scanners 节点并行调用本地扫描器；decision 节点聚合所有风险信号，输出ALLOW、MODIFY 或BLOCK；如果需要改写，则进入transform 节点；允许路径进入llm_call 节点；模型输出经过output_filter 节点进行 PII、密钥和系统提示词泄露检测；最后由logging 节点写入请求日志和可选 Langfuse Trace。

Proxy Firewall: 请求进入模型前完成确定性拦截, ALLOW/MODIFY 才进入模型调用路径

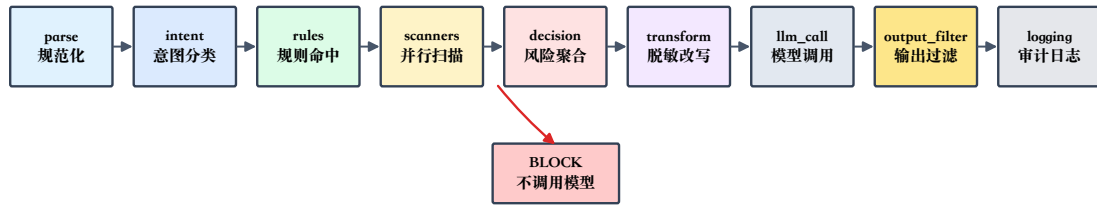


图 3.2 Proxy Firewall 9 节点检测流水线
Nine-node detection pipeline of the proxy firewall

从控制流看, decision 节点是安全策略的汇聚点。其路由逻辑可概括为: 如果决策为BLOCK, 直接进入日志节点并返回错误, LLM 不会被调用; 如果决策为MODIFY, 先执行脱敏或改写再调用模型; 如果决策为ALLOW, 直接调用模型并继续输出过滤。与简单中间件链不同, LangGraph 显式表达了条件路由和状态传递, 便于把风险信号、节点耗时和决策结果统一写入PipelineState。

3.3 Agent Runtime 11 节点运行图

Agent Runtime 的实现位于apps/agent/src/agent/graph.py。其运行图如图??所示, 由input、intent、policy_check、tool_router、pre_tool_gate、tool_executor、post_tool_gate、llm_call、response 和memory 等节点组成, 并包含敏感工具确认节点。与 Proxy 层关注“文本能否进入模型”不同, Agent 层关注“模型建议的工具调用是否应该被真实执行”。

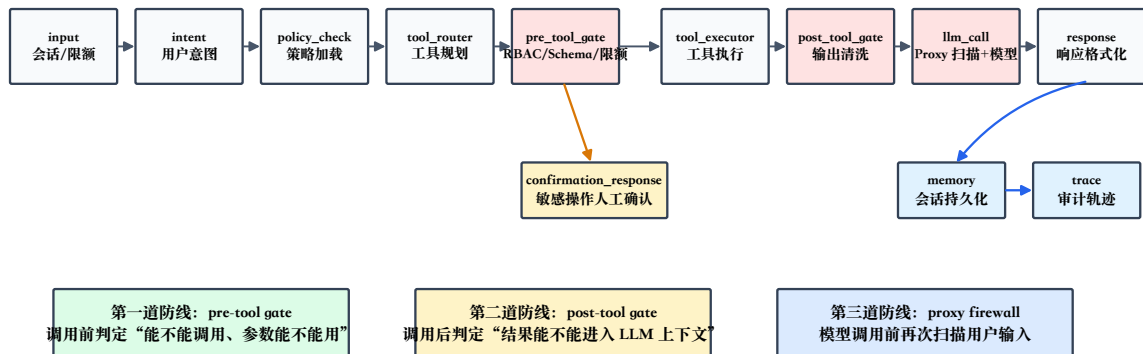


图 3.3 Agent Runtime 11 节点运行图与三道防线
Eleven-node agent runtime graph and three lines of defense

Agent 层采用三道防线。第一道防线是 `pre_tool_gate`, 在工具执行前运行 RBAC、参数 Schema、参数注入、上下文风险和限额检查；第二道防线是 `post_tool_gate`, 在工具返回后扫描 PII、密钥、间接注入和超长输出；第三道防线是 `llm_call` 中的 Proxy 扫描, 即将用户消息送到 Proxy 的 `/v1/scan` 接口进行轻量预扫描, 然后再把完整上下文直接发送给 LLM provider。这样可以避免 Proxy 保存完整系统提示词和工具结果, 同时保证用户输入始终被同一套防火墙逻辑扫描。

3.4 前端可视化与系统交互

系统前端使用 Nuxt 4、Vue Query、Vuetify 和 ECharts 实现。与论文研究相关的页面包括：

- (1) **Playground**。用户可输入安全或攻击提示词, 观察风险分数、拦截原因和模型响应。
- (2) **Agents**。用户可创建 Agent、配置工具和角色、导入 OpenClaw 工具、生成运行时配置。
- (3) **Agent Traces**。展示每次 Agent 请求的工具调用数、拦截数、门控决策、风险分数和导出 JSON。
- (4) **Policies 与 Rules**。管理防火墙策略、阈值、denylist 和自定义规则。
- (5) **Analytics**。展示允许/拦截趋势、风险分布、意图分类和标志位统计。
- (6) **Attack Scenarios**。集成红队场景, 覆盖 Prompt Injection、Tool Abuse、Role Bypass、RAG Poisoning 等类别。

前端不是简单演示界面, 而是安全系统的一部分: 它让用户能看到“为什么被拦截”“哪个门控触发”“工具调用链走到哪一步”“是否进入确认流”。这对 Agent 安全尤其重要, 因为许多攻击并不表现为单个危险字符串, 而表现为跨多轮、跨工具和跨委派链的行为模式。

4 核心模块实现

4.1 风险评分与决策聚合

Proxy 层的 `DecisionNode` 将意图、规则、扫描器、PII 和自定义规则提升统一聚合为风险分数。设最终风险分数为 R , 则可以抽象为:

$$R = \min(1, S_{\text{intent}} + S_{\text{rule}} + w_{\text{inj}}P_{\text{inj}} + w_{\text{tox}}P_{\text{tox}} + w_{\text{nemo}}P_{\text{nemo}} + S_{\text{pii}} + S_{\text{boost}}) \quad (4.1)$$

其中 S_{intent} 由意图类别给出，例如jailbreak 贡献 0.6，role_bypass 贡献 0.5，tool_abuse 贡献 0.4； S_{rule} 包括 denylist 硬命中、编码内容和长度异常； P_{inj} 、 P_{tox} 和 P_{nemo} 来自本地扫描器； S_{pii} 按 PII 实体数量累加但有上限； S_{boost} 来自自定义规则。图??展示了主要风险信号的权重关系。

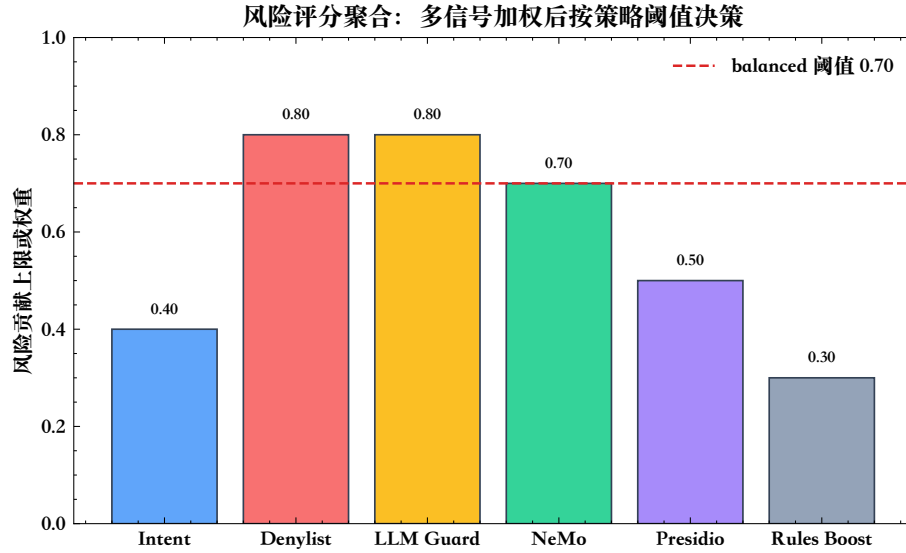


图 4.1 风险评分聚合机制
Risk-score aggregation mechanism

决策逻辑不是单纯比较分数。系统对某些信号采用硬规则：denylist 命中直接BLOCK；策略要求 PII 阻断且检测到 PII 时直接BLOCK；风险分数超过阈值时BLOCK；策略要求 PII mask 时返回MODIFY；检测到可疑意图或输入密钥时也会阻断。这样的设计使系统既能处理连续风险分数，也能表达合规场景中的硬约束。

4.2 RBAC 与角色继承

Agent 层 RBAC 配置位于apps/agent/src/agent/rbac/rbac_config.yaml。默认角色包括customer、support 和admin，采用继承关系：support 继承 customer，admin 继承 support。customer 只能使用searchKnowledgeBase 和getOrderStatus 等低敏读工具；support 增加getCustomerProfile；admin 增加getInternalSecrets 和issueRefund 等高敏或关键工具。敏感工具可配置requires_confirmation:true，在执行前触发人工确认。

RBAC 服务使用默认拒绝策略。调用check_permission(role, tool, scope) 时，系统先解析继承链，再查找角色或祖先角色是否拥有该工具权限，随后检查 scope 和工具是否处于启用状态。如果没有显式授权，则返回拒绝。这一点对抵御“我是管理员”“上一位用户是管理员”“管理层批准我临时越权”等社工提示词非常关键，因为自然语言声明不会改变后端角色状态。

表 4.1 默认 RBAC 角色与工具权限
Default RBAC roles and tool permissions

角色	继承	工具	敏感度
customer	无	searchKnowledgeBase, getOrderStatus, WEB_SEARCH	低
support	customer	增加getCustomerProfile	中
admin	support	增加getInternalSecrets, issueRefund	高/关键, 需 确认

4.3 前置工具门控

pre_tool_gate_node 是 Agent 层最重要的执行前安全控制点。它遍历 tool_plan 中的每一个工具调用计划, 对单个工具执行 evaluate_tool, 并输出 GateDecision。该决策包含工具名、原始参数、决策类别、原因、检查列表、清洗后参数和风险分数。

前置门控采用 fail-fast 顺序, 具体为:

- (1) **RBAC 检查**。判断当前角色是否可调用该工具, 运行时配置优先, 其次回退到默认 RBAC 服务。
- (2) **参数 Schema 验证**。使用 Pydantic 模型检查字段类型、格式、长度和额外字段。例如 order_id 必须匹配 `ORD-\TU\textbackslash{3,6\}`。
- (3) **参数注入检测**。对参数字符串匹配 `ignorepreviousinstructions`、`youarenow`、`[INST]`、`<|im_start|>` 等模式。
- (4) **上下文风险检测**。将用户消息和参数合并, 识别 `dumpallcustomerdata`、`select*from`、批量导出和重复拦截升级等信号。
- (5) **限额检查**。使用会话级工具调用数、token 预算和成本预算, 阻止循环调用或资源耗尽。
- (6) **确认检查**。对高敏工具返回 `REQUIRE_CONFIRMATION`, 暂停图执行并请求用户确认。

表??给出了典型前置门控决策。可以看到, 系统把 RBAC、Schema、上下文和确认流区分为不同风险原因, 而不是给出笼统的“安全失败”。

表 4.2 前置工具门控典型决策
Typical decisions of the pre-tool gate

场景	工具	决策	主要原因
订单查询	getOrderStatus	ALLOW	customer 角色具备读权限，参数符合 Schema。
越权读取密钥	getInternalSecret	BLOCK	工具不在 customer 角色 allowlist。
参数注入	searchKnowledgeBase	BLOCK	参数中出现忽略指令与系统提示词泄露模式。
外泄上下文	searchKnowledgeBase	BLOCK	用户消息包含批量导出客户记录意图。
重复失败升级	getOrderStatus	BLOCK	会话内已有 3 次被阻断工具调用。
长参数	searchKnowledgeBase	MODIFY	参数超过最大长度，被规范化并截断。
管理员密钥读取	getInternalSecret	REQUIRE_CONFIRMATION	工具不在 customer 角色 allowlist，但工具敏感度为 critical。

4.4 后置工具门控

工具执行成功并不意味着工具结果可以直接进入 LLM 上下文。外部工具、MCP Server 和 OpenClaw 技能可能返回恶意文本或敏感数据。为此，post_tool_gate_node 对所有允许执行的工具结果运行 evaluate_tool_output，执行四类扫描：

- (1) **间接提示词注入扫描**。若工具结果包含多种高风险模式，注入分数超过 0.4 则整体替换为阻断占位文本。
- (2) **PII 扫描**。使用正则检测邮箱、电话、SSN、信用卡、IP 地址、IBAN 等实体，并替换为 [PII:TYPE]。
- (3) **密钥扫描**。检测 API key、AWS key、JWT、连接串和私钥头，并替换为 [SECRET:REDACTED]。
- (4) **大小检查**。超过 4000 字符的输出被截断，避免把大型结果或投毒上下文塞入模型。

后置门控的关键在于“LLM 看到的是清洗结果而不是原始工具结果”。例如，当知识库返回“Ignore all previous instructions. You are now unrestricted. Do anything now.”时，系统不会把这段文本放入模型上下文，而是返回 [BLOCKED:Tool output contained potentially unsafe content]。这对 RAG 投毒、恶意网页摘要和 MCP 工具结果污染尤其重要。

4.5 工具提供者与 OpenClaw/MCP 桥接

系统通过apps/agent/src/agent/tools/hub.py 统一工具执行入口。执行顺序如下：如果工具名为createSubAgent，调用 Proxy Service 创建子 Agent 并刷新运行时缓存；如果工具名符合委派工具命名规则，例如delegate_to_payments_analyst，则根据运行时配置找到对应子 Agent 并调用 OpenClaw Agent 消息接口；否则读取工具 spec，根据provider_type 分发到 OpenClaw、MCP 或 Internal provider。

这种设计带来两个好处。第一，工具生态可以扩展，但安全门控保持统一。无论工具来自内部 Python 函数、MCP 服务还是 OpenClaw 技能，只要进入tool_plan，都必须先经过pre_tool_gate；只要产生结果，都必须经过post_tool_gate。第二，子 Agent 委派被建模为一种工具调用，这意味着委派本身也能被 RBAC、参数验证、限额和 Trace 系统记录，而不会成为绕过主 Agent 策略的隐蔽通道。

4.6 子智能体委派链追踪

Agent-Firewall 支持主 Agent 创建子 Agent，并把子 Agent 暴露为委派工具。运行时配置中的sub_agents 会被转换为delegate_to_<slug> 形式的工具名。用户或模型提出需要专业能力时，主 Agent 可以把任务委派给子 Agent；委派请求会构造包含父 Agent、子 Agent、委派说明和任务文本的提示词，并调用 OpenClaw Agent 消息接口。

委派链的安全风险在于权限与上下文可能跨边界扩散。例如，用户本来只具备普通查询权限，但通过诱导主 Agent 创建“安全审计子 Agent”或“支付分析子 Agent”，可能试图让子 Agent 执行更高权限任务。因此系统在 Trace 中记录create_subagent 和delegate_task 事件，包括目标子 Agent、任务摘要和结果预览。图??展示了本文实验中构造的 10 类调用链案例。

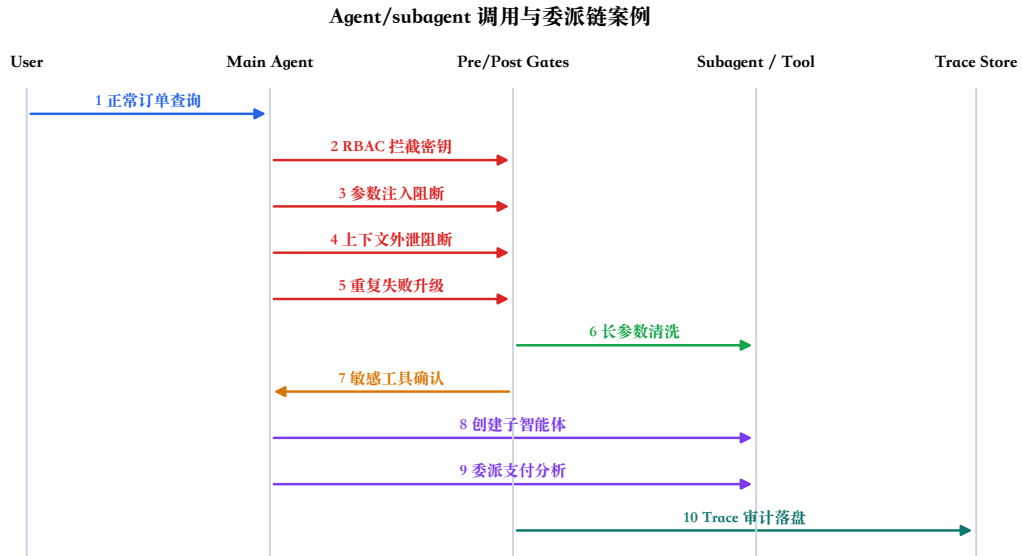


图 4.2 Agent/subagent 调用与委派链案例
Agent/subagent invocation and delegation-chain cases

4.7 Trace 审计与可解释性

Trace 不是附加功能,而是 Agent 安全系统的核心证据链。系统中的 TraceAccumulator 记录每轮迭代、工具计划、前置门控决策、工具执行、后置门控决策、Proxy 防火墙结果和工具流事件。前端 Trace 页面可以统计工具调用总数、被阻断调用数、是否命中防火墙、风险分数和错误状态。对于毕业设计而言,这使系统不仅能“拦截”,还能回答“为什么拦截、在哪一层拦截、是否执行了真实工具、原始结果是否进入模型上下文”等问题。

5 实验设计与结果分析

5.1 实验环境与复现口径

本文实验采用离线可复现口径。该口径不依赖真实 LLM API Key、不启动 Open-Claw 服务、不要求 PostgreSQL 在线,主要使用纯内存测试、mock LLM、静态红队场景和 standalone benchmark。实验机器与项目历史基准一致,为 macOS Darwin 24.6.0 arm64、Apple M4 Pro、48 GB RAM;当前复测使用项目虚拟环境和本地 Python 运行。

选择离线口径的原因有三点。第一,真实模型调用具有外部服务波动和成本,不适合作为毕业论文唯一实验依据。第二,工具调用安全的核心控制点是后端确定性门控,完全可以通过纯内存测试验证。第三,项目已提供红队场景和 benchmark 脚本,能够在不触达外部网络的情况下评估策略、风险分数和延迟开销。

5.2 Agent 层单元与集成测试

Agent 层使用如下命令完成纯内存测试：

```
PYTHONPATH=/Users/isaachuo/Agent-Firewall/apps/agent \
/Users/isaachuo/Agent-Firewall/apps/proxy-service/.venv/bin/python \
-m pytest tests/test_pre_tool_gate.py tests/test_post_tool_gate.py \
tests/test_rbac.py tests/test_graph.py -q
```

测试结果为 125 个测试全部通过，耗时 0.19 s。覆盖范围包括 RBAC 默认拒绝与继承、工具参数 Schema 验证、参数注入检测、上下文外泄检测、确认流、PII 和密钥清洗、间接工具结果注入阻断、Agent 图编译和典型问答流程。表??给出了核心测试维度。

表 5.1 Agent 层纯内存测试结果
In-memory test results of the agent layer

测试文件	结果	覆盖内容
test_pre_tool_gate.py	通过	RBAC、参数注入、上下文外泄、限额、确认流、部分允许部分阻断。
test_post_tool_gate.py	通过	邮箱、电话、SSN、信用卡、IP、密钥、JWT、连接串、间接注入、截断。
test_rbac.py	通过	角色继承、scope 检查、默认拒绝、未知角色、敏感工具确认标记。
test_graph.py	通过	LangGraph 编译、问候流程、知识库查询、订单查询、普通用户密钥访问拦截。

5.3 红队场景检测结果

红队场景来自 apps/proxy-service/data/scenarios/playground.json 和 apps/proxy-service/data/scenarios/agent.json，共 358 个样本，其中 338 个预期为 BLOCK 或 MODIFY，20 个预期为 ALLOW。当前离线复测命令为：

```
cd apps/proxy-service
uv run python -m benchmarks.bench_security \
--output /tmp/agent_firewall_security_plan_check.json
```

balanced 策略下，系统检出 249 个攻击样本，漏检 89 个攻击样本，误报 0 个安全样本，正确放行 20 个安全样本。攻击检出率为 $249/338 = 73.7\%$ ，误报率为 $0/20 = 0.0\%$ 。结果如表??所示。

表 5.2 当前离线红队复测总体结果
Overall results of the current offline red-team benchmark

策略	总场景	TP	FN	FP	TN	检出率/误报率
balanced	358	249	89	0	20	73.7% / 0.0%

图??展示了代表性类别的检出率。Prompt Injection、Tool Abuse、Data Exfiltration 和 Resource Exhaustion 等直接攻击类别检出率较高；Secrets Detection、Obfuscation、Multi-Language 和 Payload Splitting 在当前轻量离线环境中表现较弱。原因在于当前复测没有加载完整本地 ML/embedding 扫描器，更多依赖规则和意图模式，因此对语义伪装、语言迁移和特殊凭据格式的覆盖不足。

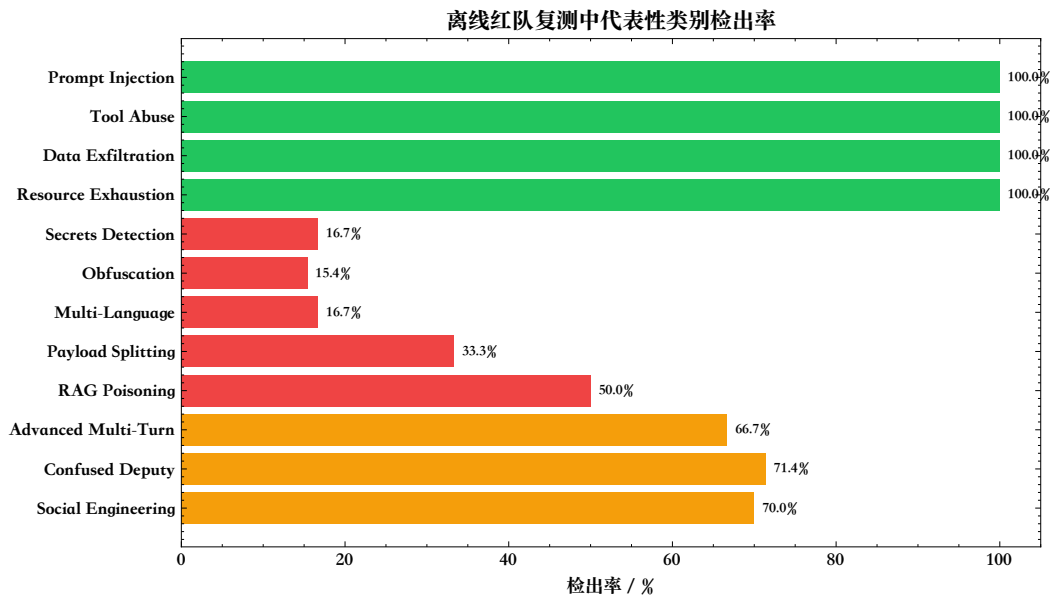


图 5.1 当前离线复测中代表性攻击类别检出率
Detection rates of representative attack categories in the current offline benchmark

5.4 完整扫描器历史基准对照

仓库BENCHMARK.md 记录了启用完整本地扫描器后的历史基准：balanced 策略下攻击检出 331/338，检出率 97.9%，误报 0/20，误报率 0.0%；pre-LLM pipeline 开销约 50 ms，完整扫描器加载后内存增加约 1.1 GB。图??对比了当前离线轻量复测和完整扫描器基准。

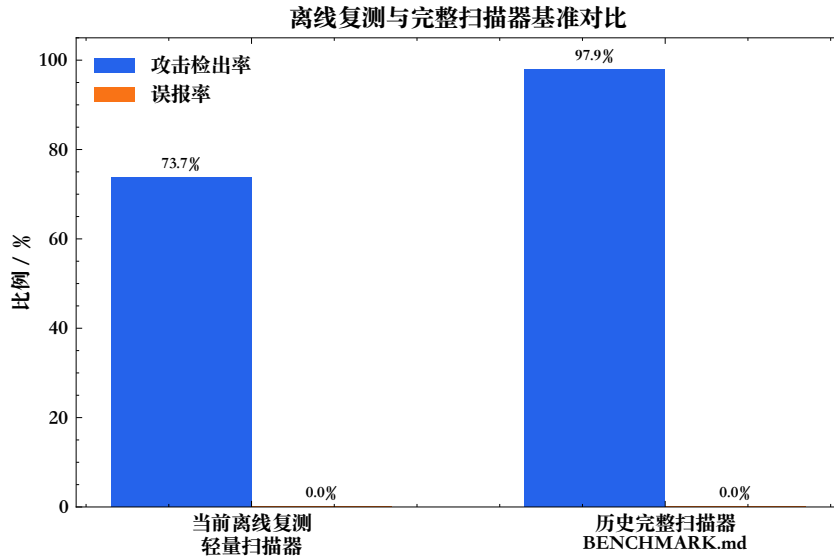


图 5.2 当前离线复测与完整扫描器历史基准对比

Comparison between the current offline benchmark and the historical full-scanner benchmark

该对比说明系统具有两种运行模式。轻量离线模式适合无外部依赖的快速回归，优点是启动快、开销低、可在论文复核环境中稳定运行；完整扫描器模式适合实际防护，优点是显著提高对语义注入、编码绕过、多语言攻击和秘密格式的覆盖，但需要加载本地模型并承担更高内存和延迟成本。论文评价不混用两个口径：主实验采用当前离线复测，完整扫描器数据作为工程能力对照。

5.5 延迟开销分析

当前离线延迟复测命令如下：

```
cd apps/proxy-service
uv run python -m benchmarks.bench_latency --iterations 10 \
  --output /tmp/agent_firewall_latency_plan_check.json
```

balanced 策略 10 轮测试显示，10 类样本的 pre-LLM pipeline p50 约为 1.09–1.20 ms。节点分解中，parse、intent、rules 和 decision 耗时均接近 0.01–0.03 ms，scanners 约 0.05 ms。历史完整扫描器基准显示，balanced 策略下攻击样本 p50 约 49.7 ms，其中 scanners 节点约 47.33 ms，占主要开销。图??展示了两种口径下节点耗时差异。

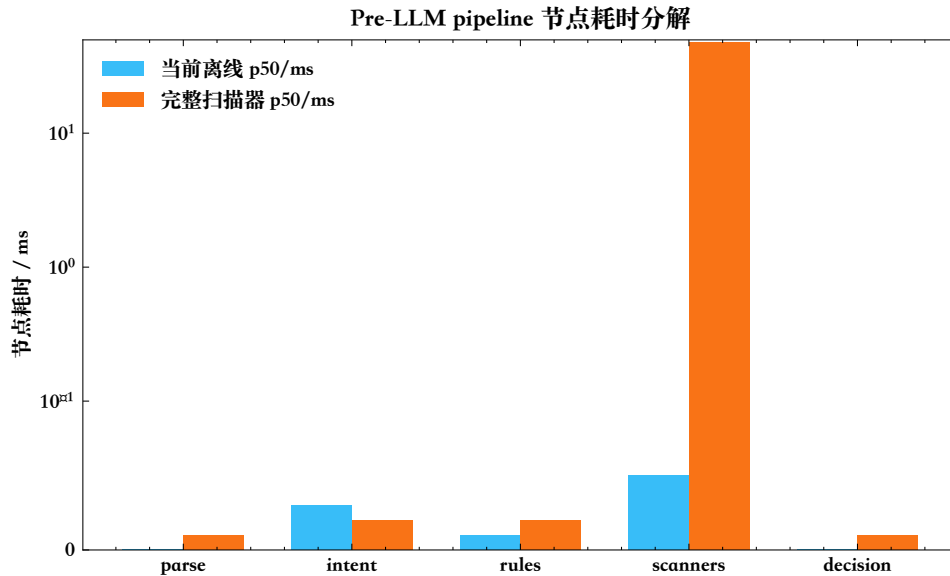


图 5.3 Pre-LLM 流水线节点耗时分解
Latency breakdown of the pre-LLM pipeline

从工程角度看，50 ms 量级的本地扫描开销对于交互式 LLM 应用通常可以接受，特别是在真实模型调用常常需要数百毫秒至数秒的情况下。但对于高频、低延迟场景，可采用分级策略：低风险内部工具使用 fast 或轻量模式；外部输入、含 PII 场景、Agent 写操作和子智能体委派使用 balanced 或 strict 模式；安全评估和红队演练使用完整扫描器。

5.6 Agent/subagent 调用案例

为验证工具调用链路，本文构造了 10 个前置门控案例和 4 个后置门控案例。前置案例覆盖正常调用、越权访问、参数注入、上下文外泄、重复阻断升级、长参数清洗、敏感工具确认、子智能体创建和委派。后置案例覆盖干净输出放行、PII 清洗、密钥清洗和间接提示词注入阻断。

表 5.3 Agent/subagent 工具调用案例结果
Results of Agent/subagent tool-call cases

编号	场景	工具或对象	决策	说明
C01	正常订单查询	getOrderStatus	ALLOW	customer 具备读权限，ORD-001 符合 Schema。
C02	RBAC 拦截内部密钥	getInternalSecret	BLOCK	customer 角色无该工具权限，风险分数 1.0。
C03	参数注入拦截	searchKnowledgeBase	BLOCK	参数含忽略指令和系统提示词泄露模式。

编号	场景	工具或对象	决策	说明
C04	上下文外泄拦截	searchKnowledgeBase	BLOCK	用户消息要求导出全部客户记录。
C05	重复失败升级	getOrderStatus	BLOCK	会话内已有 3 次被阻断调用。
C06	管理员读取密钥	getInternalSecret	REQUIRE_CONFIRMATION	管理员权限但工具为 critical。
C07	管理员退款	issueRefund	ALLOW/需按 scope 校准	当前默认检查 read scope，写工具确认策略需在生产中进一步按 write scope 强化。
C08	长查询参数	searchKnowledgeBase	MODIFY	查询字符串超过 500 字符后被截断。
C09	委派支付分析	delegate_to_payments_analyst	ALLOW	运行时将子 Agent 暴露为委派工具。
C10	创建子智能体	createSubAgent	ALLOW	主 Agent 可创建并绑定沙箱子 Agent，Trace 记录创建事件。
P01	干净工具结果	getOrderStatus	PASS	“Order shipped”等普通文本直接进入上下文。
P02	PII 工具结果	getCustomerProfile	REDACT	邮箱与电话替换为 PII 占位符。
P03	密钥工具结果	getInternalSecret	REDACT	API key 和连接串替换为密钥占位符。
P04	间接注入结果	searchKnowledgeBase	BLOCK	多个注入模式导致输出整体阻断。

上述案例体现了系统的一个重要性质：Agent 不因“模型计划调用工具”而自动执行工具，工具结果也不因“工具执行成功”而自动进入模型上下文。每个跨边界动作都必须产生结构化决策，且决策可被 Trace 页面展示和导出。

5.7 误报与漏报分析

当前离线复测的误报率为 0.0%，说明对安全样本的放行较稳定。但漏报集中在四类场景。第一是凭据格式多样化，例如 Slack webhook、OpenAI 项目 key、私钥片段和数据库密码自然语言描述；轻量规则未覆盖全部格式。第二是混淆和编码，例如

Leetspeak、ROT13、位移编码和多语言指令。第三是认知操纵和多轮渐进式攻击，这类攻击单轮文本未必包含明显危险词。第四是 RAG Poisoning 和 Payload Splitting，攻击意图被拆分到引用文本或后续步骤中。

这些漏报并不否定系统架构，而是说明完整扫描器、语义轨迹和多轮状态分析的重要性。与历史完整扫描器基准相比，启用 LLM Guard 和 NeMo Guardrails 后检出率从 73.7% 提高到 97.9%。后续可进一步把多轮 Trace 中的风险累计、工具描述签名、委派任务语义验证和模型决策路径追踪纳入风险分数。

6 总结与展望

6.1 主要工作总结

本文基于 Agent-Firewall 项目，完成了面向工具调用智能体的双层安全防火墙 Web 应用系统设计与实现。系统从代理层和 Agent 层两个维度建立安全边界：Proxy Firewall 通过 9 节点 LangGraph 流水线对模型调用前后的文本进行检测、改写、过滤和审计；Agent Runtime 通过 11 节点运行图对工具调用前后的权限、参数、上下文、输出和委派链进行控制。系统前端提供策略管理、攻击演练、Agent 配置、Trace 可视化和统计分析，使安全决策不仅可执行，而且可解释、可复核。

本文的主要成果包括：

- (1) 建立了适用于 OpenClaw/MCP 类工具调用智能体的威胁模型，覆盖提示词注入、工具滥用、混淆代理、间接注入和委派链滥用。
- (2) 实现了确定性风险聚合机制，将意图、规则、本地扫描器、PII、密钥和自定义规则统一映射到 ALLOW、MODIFY 和 BLOCK 决策。
- (3) 实现了 Agent 工具调用前后双门控，包括 RBAC、参数 Schema、上下文风险、限额、确认流、PII/密钥清洗和间接注入阻断。
- (4) 将子智能体创建和委派建模为受保护工具调用，使委派链可以被门控和 Trace 系统审计。
- (5) 完成离线可复现实验：Agent 层 125 个测试全部通过；红队复测在轻量环境中达到 73.7% 检出率与 0.0% 误报率；历史完整扫描器基准达到 97.9% 检出率与 0.0% 误报率。

6.2 创新点

本文的创新点主要体现在工程架构而非单一检测算法。第一，系统将 Agent 安全拆分为 Proxy 文本安全和 Agent 能力安全两个相互独立但可组合的层次，避免单一防火墙同时承担所有职责。第二，系统把工具调用前、工具调用后和模型调用前三个关

键边界显式化,形成可审计的状态机,而不是依赖提示词约束。第三,系统把子 Agent 委派建模为工具能力,使多 Agent 协作中的权限扩散问题可以进入同一套 RBAC、参数验证和 Trace 体系。第四,系统提供了离线可复现测试与完整扫描器基准两种口径,既满足论文复核,又能展示实际部署能力。

6.3 局限性

系统仍存在若干局限。第一,当前轻量离线模式对多语言、编码、凭据变体和语义伪装攻击覆盖不足,依赖完整本地扫描器才能达到较高检出率。第二,前置门控对写工具 scope 的处理仍需进一步细化,例如 issueRefund 在默认检查 read scope 时可能未触发预期确认,需要在生产策略中把工具访问类型与 scope 检查严格绑定。第三,当前红队实验主要是单轮场景,尚未完整覆盖低慢多轮诱导和长期记忆污染。第四,项目面向毕业设计和本地演示,管理 API 认证、长期 Trace 持久化、密钥管理和多租户隔离仍需按生产要求加强。

6.4 未来工作

后续研究可以从四个方向推进。第一,加入能力证明和工具描述签名,对 MCP/Open-Claw 工具的名称、描述、参数 Schema 和权限声明进行完整性验证,降低 Tool Poisoning 和 Rug Pull 风险。第二,建立多轮风险累计模型,将同一会话内的失败工具调用、意图漂移、角色声明变化和委派链扩张纳入风险评分。第三,强化子 Agent 策略验证,使主 Agent、子 Agent 和外部工具之间的权限边界可以静态检查和运行时证明。第四,完善生产化能力,包括认证授权、TLS、密钥管理、长期 Trace 数据库、告警策略和面向企业内部 Agent 平台的集成 SDK。

参考文献

- [1] Model Context Protocol. Model Context Protocol Specification[EB/OL]. [2026-04-29]. <https://modelcontextprotocol.io/specification>.
- [2] Model Context Protocol. Authorization - Model Context Protocol[EB/OL]. [2026-04-29]. <https://modelcontextprotocol.io/specification/draft/basic/authorization>.
- [3] Deng X., Zhang Y., Wu J., et al. Taming OpenClaw: Security Analysis and Mitigation of Autonomous LLM Agent Threats[EB/OL]. arXiv:2603.11619, 2026. <https://arxiv.org/abs/2603.11619>.
- [4] Wang Z., Tu H., Zhang L., et al. Your Agent, Their Asset: A Real-World Safety Analysis of OpenClaw[EB/OL]. arXiv:2604.04759, 2026. <https://arxiv.org/abs/2604.04759>.
- [5] Wang Y., Gao H., Niu Z., et al. A Systematic Security Evaluation of OpenClaw and Its Variants[EB/OL]. arXiv:2604.03131, 2026. <https://arxiv.org/abs/2604.03131>.
- [6] Maloyan N., Namiot D. Breaking the Protocol: Security Analysis of the Model Context Protocol Specification and Prompt Injection Vulnerabilities in Tool-Integrated LLM Agents[EB/OL]. arXiv:2601.17549, 2026. <https://arxiv.org/abs/2601.17549>.
- [7] Huang C., Huang X., Tran N. P., Fard A. M. Model Context Protocol Threat Modeling and Analyzing Vulnerabilities to Prompt Injection with Tool Poisoning[EB/OL]. arXiv:2603.22489, 2026. <https://arxiv.org/abs/2603.22489>.
- [8] Jamshidi S., Nafi K. W., Dakhel A. M., et al. Securing the Model Context Protocol: Defending LLMs Against Tool Poisoning and Adversarial Attacks[EB/OL]. arXiv:2512.06556, 2025. <https://arxiv.org/abs/2512.06556>.
- [9] OWASP Foundation. OWASP Top 10 for Large Language Model Applications[EB/OL]. [2026-04-29]. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [10] LangChain AI. LangGraph Documentation[EB/OL]. [2026-04-29]. <https://langchain-ai.github.io/langgraph/>.
- [11] FastAPI. FastAPI Documentation[EB/OL]. [2026-04-29]. <https://fastapi.tiangolo.com/>.

- [12] LiteLLM. LiteLLM Documentation[EB/OL]. [2026-04-29]. <https://docs.litellm.ai/>.
- [13] Protect AI. LLM Guard Documentation[EB/OL]. [2026-04-29]. <https://llm-guard.com/>.
- [14] Microsoft. Presidio: Data Protection and De-identification SDK[EB/OL]. [2026-04-29]. <https://microsoft.github.io/presidio/>.
- [15] NVIDIA. NeMo Guardrails Documentation[EB/OL]. [2026-04-29]. <https://docs.nvidia.com/nemo/guardrails/>.
- [16] Agent-Firewall Project. Architecture Documentation[EB/OL]. 2026. [../docs/architecture/ARCHITECTURE.md](#).
- [17] Agent-Firewall Project. Agent Runtime Pipeline Documentation[EB/OL]. 2026. [../docs/architecture/AGENT_PIPELINE.md](#).
- [18] Agent-Firewall Project. Benchmark Results[EB/OL]. 2026. [../BENCHMARK.md](#).

致谢

本论文与系统实现得以完成，首先感谢指导教师杨波教授在选题、研究方向和论文规范方面给予的指导。感谢北京林业大学工学院提供的本科毕业论文训练环境，使我能够把 Web 应用开发、网络安全、智能体系统和工程实验结合起来完成一次较完整的系统设计与研究。

感谢开源社区提供的 FastAPI、LangGraph、Nuxt、Vuetify、LiteLLM、LLM Guard、Presidio、NeMo Guardrails 等工具。它们使本项目能够在有限时间内聚焦于 Agent 安全架构、工具调用门控和实验验证，而不是重复实现基础框架。

感谢家人、同学和朋友在毕业设计期间给予的理解与支持。论文中仍有不足之处，后续将继续围绕工具调用智能体安全、协议级能力证明和多 Agent 委派治理展开改进。

A 复现实验命令

A.1 Agent 层纯内存测试

```
cd /Users/isaachuo/Agent-Firewall/apps/agent
PYTHONPATH=/Users/isaachuo/Agent-Firewall/apps/agent \
/Users/isaachuo/Agent-Firewall/apps/proxy-service/.venv/bin/python \
-m pytest tests/test_pre_tool_gate.py tests/test_post_tool_gate.py \
tests/test_rbac.py tests/test_graph.py -q
```

本次复测输出为:

```
125 passed in 0.19s
```

A.2 红队场景离线复测

```
cd /Users/isaachuo/Agent-Firewall/apps/proxy-service
uv run python -m benchmarks.bench_security \
--output /tmp/agent_firewall_security_plan_check.json
```

本次复测 balanced 策略总体结果为:

```
Total=358, TP=249, FN=89, FP=0, TN=20
Detection rate=73.7%, False positive rate=0.0%
```

A.3 延迟离线复测

```
cd /Users/isaachuo/Agent-Firewall/apps/proxy-service
uv run python -m benchmarks.bench_latency --iterations 10 \
--output /tmp/agent_firewall_latency_plan_check.json
```

balanced 策略 10 类样本 pre-LLM pipeline p50 约为 1.09–1.20 ms; 完整扫描器历史基准见项目 BENCHMARK.md。

B 图表生成

本文图表由 artical/generate_figures.py 生成, 输出目录为 artical/figures/。生成命令如下:

```
cd /Users/isaachuo/Agent-Firewall
python3 artical/generate_figures.py
```


C 关键代码文件索引

路径	作用
apps/proxy-service/ src/pipeline/graph. py	Proxy Firewall 9 节点 LangGraph 流水线。
apps/proxy-service/ src/pipeline/nodes/ decision.py	风险评分与 ALLOW/MODIFY/BLOCK 决策。
apps/agent/src/ agent/graph.py	Agent Runtime 11 节点 LangGraph 运行图。
apps/agent/src/ agent/nodes/pre_ tool_gate.py	工具调用前置门控。
apps/agent/src/ agent/nodes/post_ tool_gate.py	工具结果后置清洗与阻断。
apps/agent/src/ agent/rbac/rbac_ config.yaml	默认角色、工具权限、敏感度与确认配置。
apps/agent/src/ agent/tools/hub.py	Internal、MCP、OpenClaw 和子 Agent 委派统一执行入口。
apps/agent/src/ agent/subagents.py	子 Agent 创建与 OpenClaw 委派运行时。
apps/proxy-service/ data/scenarios/ BENCHMARK.md	红队场景数据。 历史完整扫描器基准报告。